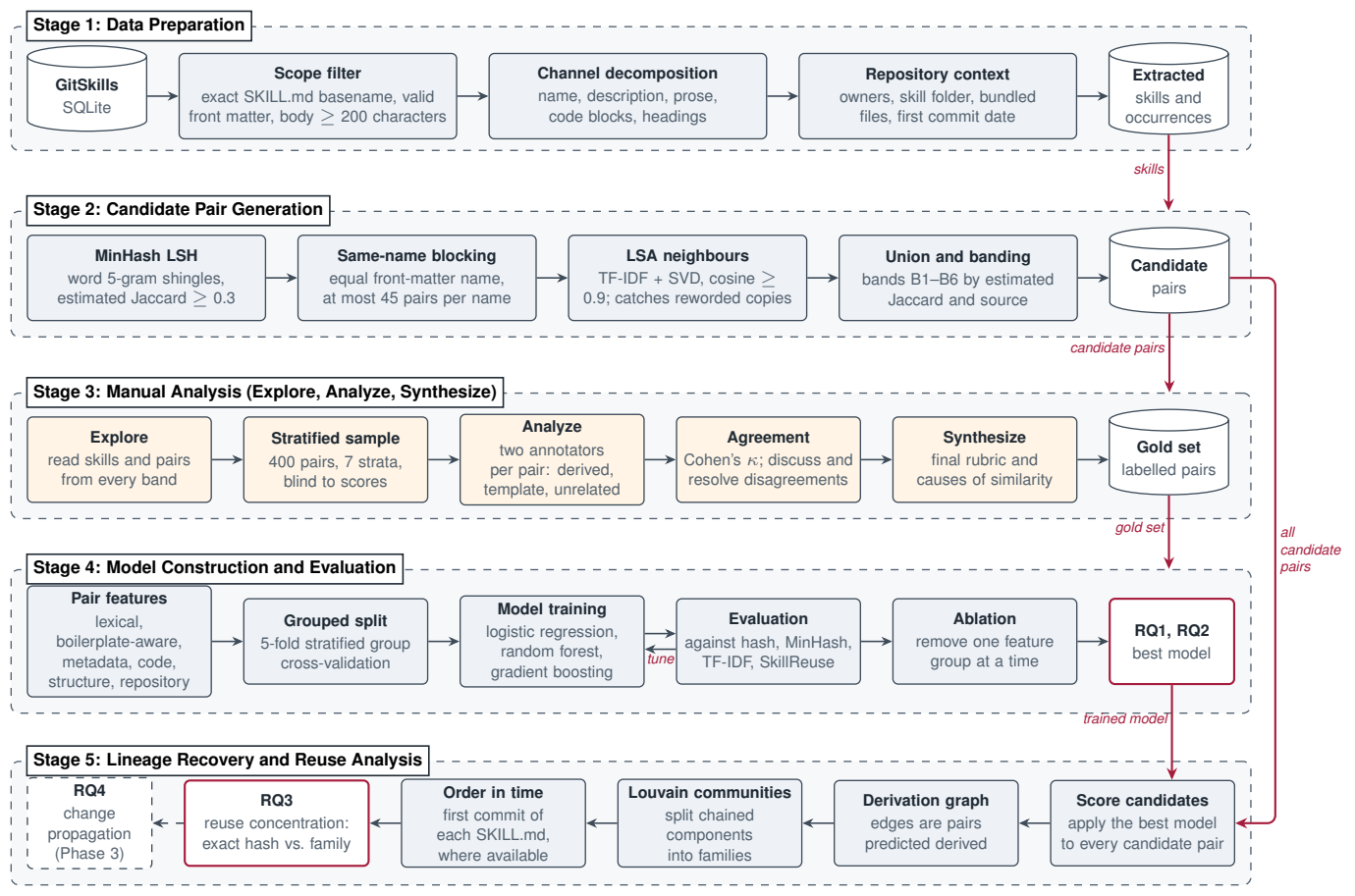
Stage 1: Data Preparation
GitSkills
SQLite
Scope filter
exact SKILL.md basename, valid front matter, body ≥ 200 characters
Channel decomposition
name, description, prose, code blocks, headings
Repository context
owners, skill folder, bundled files, first commit date
Extracted
skills and occurrences
skills
Stage 2: Candidate Pair Generation
MinHash LSH
word 5-gram shingles, estimated Jaccard ≥ 0.3
Same-name blocking
equal front-matter name, at most 45 pairs per name
LSA neighbours
TF-IDF + SVD, cosine ≥ 0.9; catches reworded copies
Union and banding
bands B1–B6 by estimated Jaccard and source
Candidate
pairs
candidate pairs
Stage 3: Manual Analysis (Explore, Analyze, Synthesize)
Explore
read skills and pairs from every band
Stratified sample
400 pairs, 7 strata, blind to scores
Analyze
two annotators per pair: derived, template, unrelated
Agreement
Cohen's κ; discuss and resolve disagreements
Synthesize
final rubric and causes of similarity
Gold set
labelled pairs
gold set
all candidate pairs
Stage 4: Model Construction and Evaluation
Pair features
lexical, boilerplate-aware, metadata, code, structure, repository
Grouped split
5-fold stratified group cross-validation
Model training
logistic regression, random forest, gradient boosting
tune
Evaluation
against hash, MinHash, TF-IDF, SkillReuse
Ablation
remove one feature group at a time
RQ1, RQ2
best model
trained model
Stage 5: Lineage Recovery and Reuse Analysis
RQ4
change propagation (Phase 3)
RQ3
reuse concentration: exact hash vs. family
Order in time
first commit of each SKILL.md, where available
Louvain communities
split chained components into families
Derivation graph
edges are pairs predicted derived
Score candidates
apply the best model to every candidate pair